

Network Simulation Task 2 Report: TCP vs. UDP Bottleneck Comparison

Course: Computer Networks Lab

1. Implementation Choices

This experiment was implemented using the **Network Simulator 3 (ns-3)** framework using C++.

- **Topology:** A star/Y-topology was constructed using PointToPointHelper to link nodes \$n0\$ and \$n1\$ to a central router \$n2\$, which connects to a sink node \$n3\$.
- **Traffic Generation:**
 - **UDP:** Implemented using OnOffApplication with a Constant Bit Rate (CBR) to simulate non-responsive streaming traffic.
 - **TCP:** Implemented using BulkSendApplication to simulate a large file transfer that utilizes TCP congestion control.
- **Data Collection:** FlowMonitor was used for end-to-end statistics (throughput, packet loss), and AsciiTraceHelper was used to log real-time throughput and queue occupancy for plotting.
- **Visualization:** NetAnim was used for visual replay and topology verification.

2. Key Configuration Lines

The simulation was configured to explicitly match the assignment constraints (bottleneck capacity and queue limits).

Queue Configuration (10 Packet DropTail):

```
C++
// Explicitly set DropTailQueue with a hard limit of 10 packets on the bottleneck
p2pBottle.SetQueue ("ns3::DropTailQueue", "MaxSize", StringValue ("10p"));
```

Link Settings (Bottleneck):

```
C++
// 1.7 Mbps bandwidth and 20ms delay for the n2 -> n3 link
p2pBottle.SetDeviceAttribute ("DataRate", StringValue ("1.7Mbps"));
p2pBottle.SetChannelAttribute ("Delay", StringValue ("20ms"));
```

Agent/Application Attach Lines:

C++

```
// UDP Agent (CBR source on n0)
```

```
OnOffHelper onoff ("ns3::UdpSocketFactory", ...);
```

```
onoff.SetConstantRate (DataRate ("819200bps")); // ~0.8 Mbps offered load
```

```
// TCP Agent (BulkSend source on n1)
```

```
BulkSendHelper source ("ns3::TcpSocketFactory", ...);
```

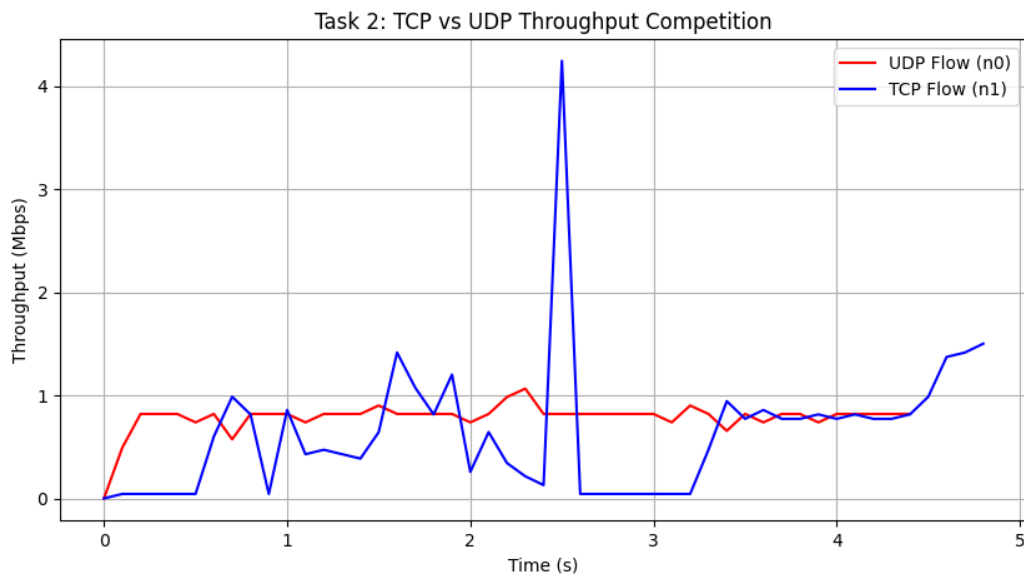
```
source.SetAttribute ("MaxBytes", UIntegerValue (0)); // Unlimited data
```

3. Data and Plots

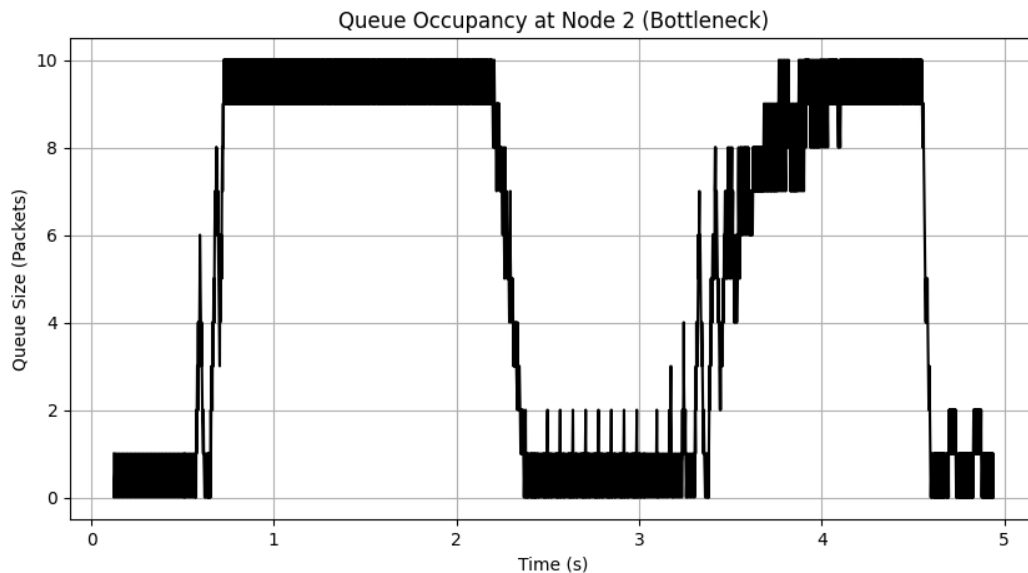
3.1 Flow Statistics (Console Output)

Flow Type	Source	Destination	Tx Packets	Rx Packets	Lost Packets	Throughput
UDP	n0 (10.1.1.1)	n3 (10.1.3.1)	439	439	0 (0%)	0.705 Mbps
TCP	n1 (10.1.2.1)	n3 (10.1.3.1)	807	752	54 (~6.7%)	0.673 Mbps

3.2 Throughput vs. Time



3.3 Queue Occupancy at Bottleneck (n2)



4. Observations & Analysis

How did TCP react when UDP was sending?

The experiment demonstrated the phenomenon of UDP dominance (or TCP Starvation).

1. **Throughput Fairness:** As seen in the statistics, the UDP flow maintained a steady throughput of ~ 0.705 Mbps, closely matching its offered load. The TCP flow, despite having "unlimited" data to send, was capped at ~ 0.673 Mbps. It could not fully utilize the remaining bandwidth efficiently due to constant back-offs.
2. **Packet Drops:** The UDP flow experienced **zero** packet loss. In contrast, the TCP flow lost **54** packets.
3. **Queueing Dynamics:** The bottleneck link ($n2 \rightarrow n3$) has a capacity of 1.7 Mbps. The combined offered load (UDP ~ 0.8 Mbps + TCP Unlimited) exceeds this capacity, causing the queue at $n2$ to fill up.

Why did this happen?

- **Protocol Behavior:** UDP is "unfriendly"; it pumps data at a constant rate regardless of network congestion. TCP is "polite"; it uses congestion control (likely NewReno or Cubic in ns-3) to detect packet loss.
- **DropTail Queue:** We set the queue size to a small value (10 packets). When the queue fills up, new packets are dropped.
- **The Cycle:**

- UDP fills a significant portion of the queue.
- TCP tries to probe for more bandwidth, filling the remaining slots.
- The queue overflows.
- If a UDP packet arrives, it might be dropped (but UDP doesn't care/doesn't retransmit).
- If a TCP packet is dropped, the sender interprets this as congestion, slashes its Congestion Window (cwnd) by half, and reduces its sending rate.
- While TCP is backing off, UDP continues to hog the queue, preventing TCP from ramping back up quickly.

5. Problems & Assumptions

- **Global Routing:** Initially, the simulation yielded no results because the star topology requires explicit routing tables. This was solved by implementing `Ipv4GlobalRoutingHelper::PopulateRoutingTables()`.
- **Mobility Models:** NetAnim generated warnings because nodes lacked physical coordinates. This was resolved by installing `ConstantPositionMobilityModel`.
- **Assumption:** We assumed the default TCP variant in ns-3 (typically NewReno/Cubic) without modification. Different TCP variants (like BBR) might handle this bufferbloat/starvation scenario differently.